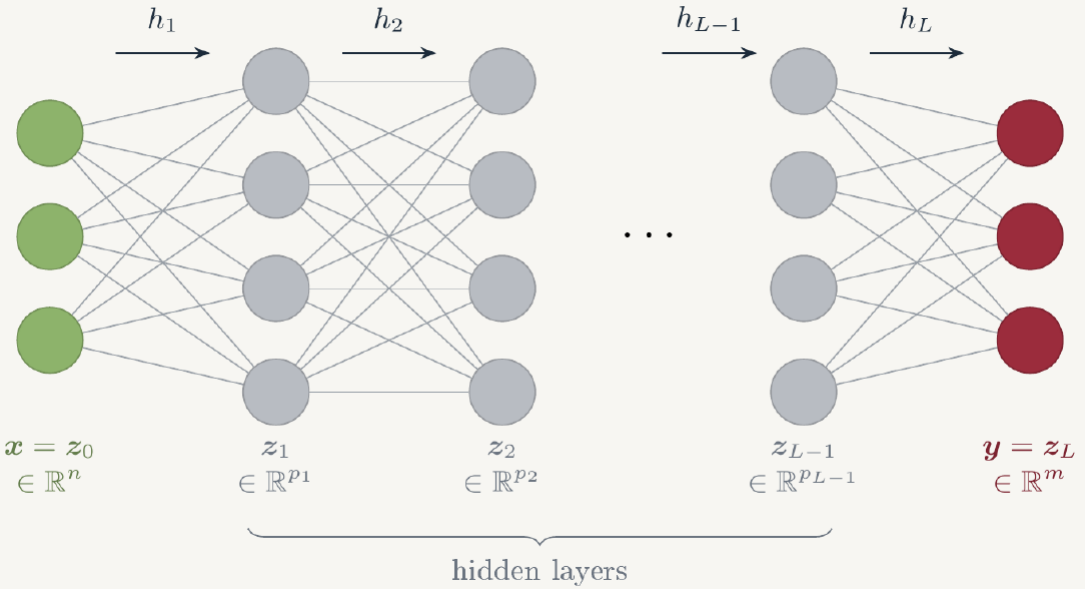
PINNS COURSE · PART 1

Introduction to Machine Learning: Supervised Learning

Neural networks, losses and optimizers

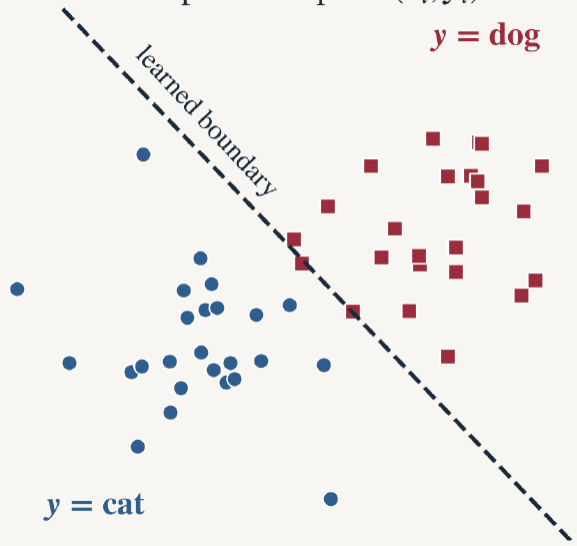
Nassim RAZAALY

v1 · 14/10/26

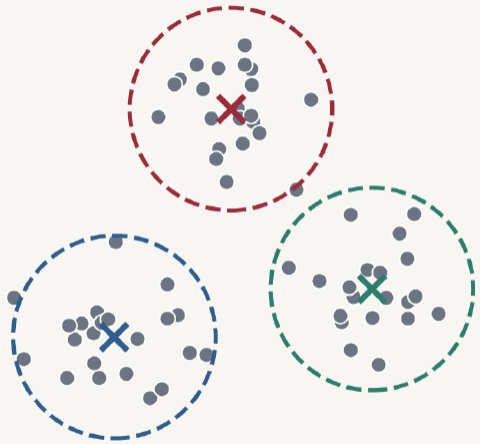


Three ways to learn

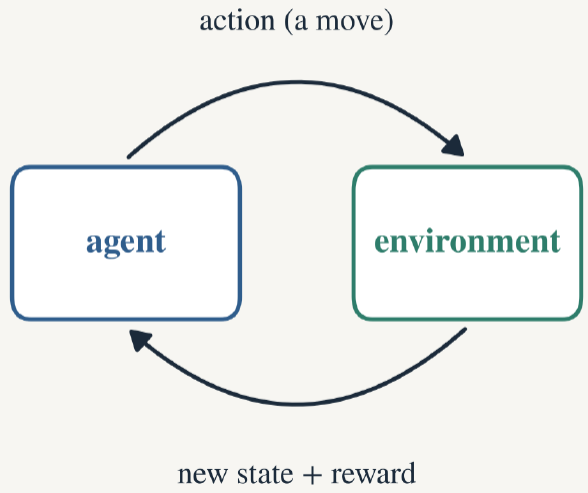
supervised: pairs (x_i, y_i)



unsupervised: x_i only, find structure



reinforcement: actions and rewards



Learning	Data	Goal	Examples
Supervised	pairs ($\mathbf{x}_i$, $\mathbf{y}_i$)	predict **y** for a new **x**	image labels, translation, weather, **PINNs**
Unsupervised	$\mathbf{x}_i$ only	find structure, groups	similar songs or listeners, anomaly detection
Reinforcement	actions, rewards	a strategy with high reward	AlphaGo, robotics, ChatGPT

This course: neural networks for supervised learning

I · PROBLEM
data $(\mathbf{x}_i, \mathbf{y}_i)$

→

II · MODEL
MLP $\tilde{f}$ with parameters $\boldsymbol{\theta}$

→

III · TRAINING
loss $\mathcal{L}(\boldsymbol{\theta})$, data split

→

IV · OPTIMIZATION
gradients, Adam, L-BFGS

→

V · IN PRACTICE
PyTorch, a recipe

Why neural networks?

- they can approximate any continuous function (part II);
- their parameters are found by gradient methods, at any scale (part IV);
- they are differentiable with respect to their inputs: derivatives $\partial\tilde{u}/\partial x$, $\partial\tilde{u}/\partial t$ are computed exactly by automatic differentiation, which is what PINNs need.

Why the MLP?

The simplest network: layers of neurons, fully connected. It is the standard model of PINNs, and the building block of larger architectures.

Notation: vectors in bold ($\mathbf{x}$, $\mathbf{y}$, $\boldsymbol{\theta}$), N samples, d parameters, L layers.

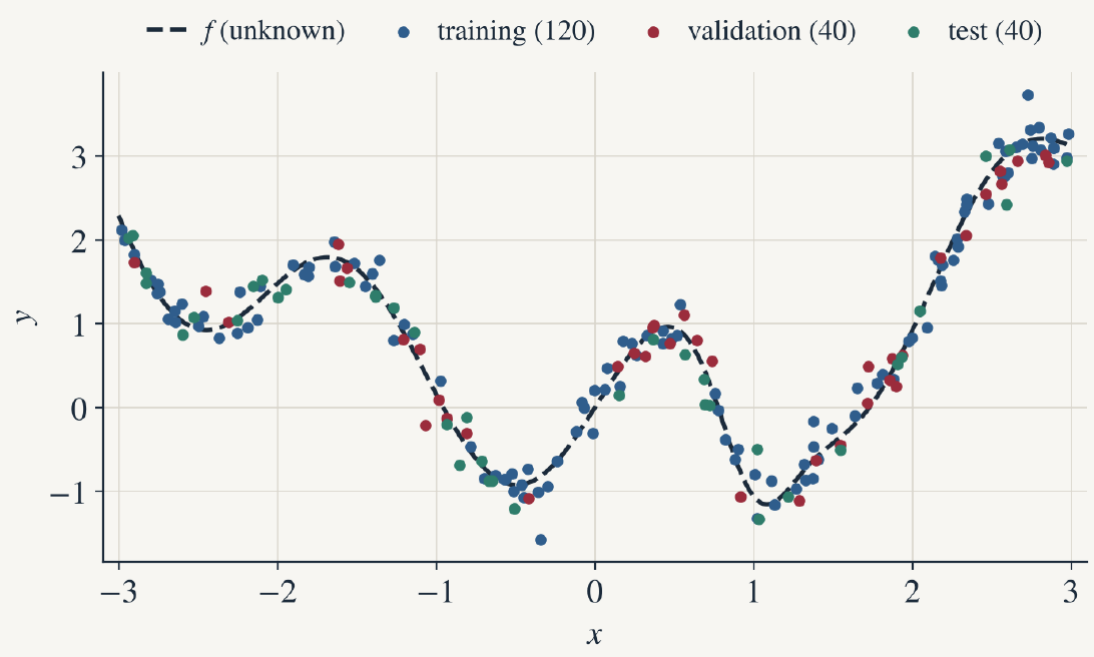
I. Learning a function from input/output pairs

$f: \mathbb{R}^n \to \mathbb{R}^m, \qquad \boldsymbol{x} \mapsto \boldsymbol{y} = f(\boldsymbol{x})$ unknown

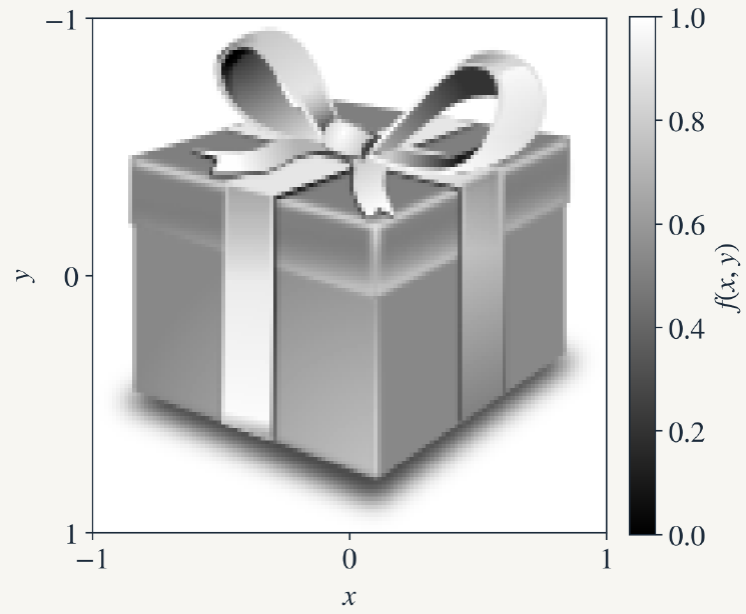
$\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N, \qquad \boldsymbol{y}_i = f(\boldsymbol{x}_i) + \boldsymbol{\varepsilon}_i$ (noise)

find $\boldsymbol{\theta} \in \mathbb{R}^d$ such that $\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx f(\boldsymbol{x})$

Example 1 · n = m = 1, N = 200 noisy samples



Example 2 · an image is a function



f : [−1, 1]² → [0, 1], n = 2, m = 1, N = 128² = 16 384 pixels

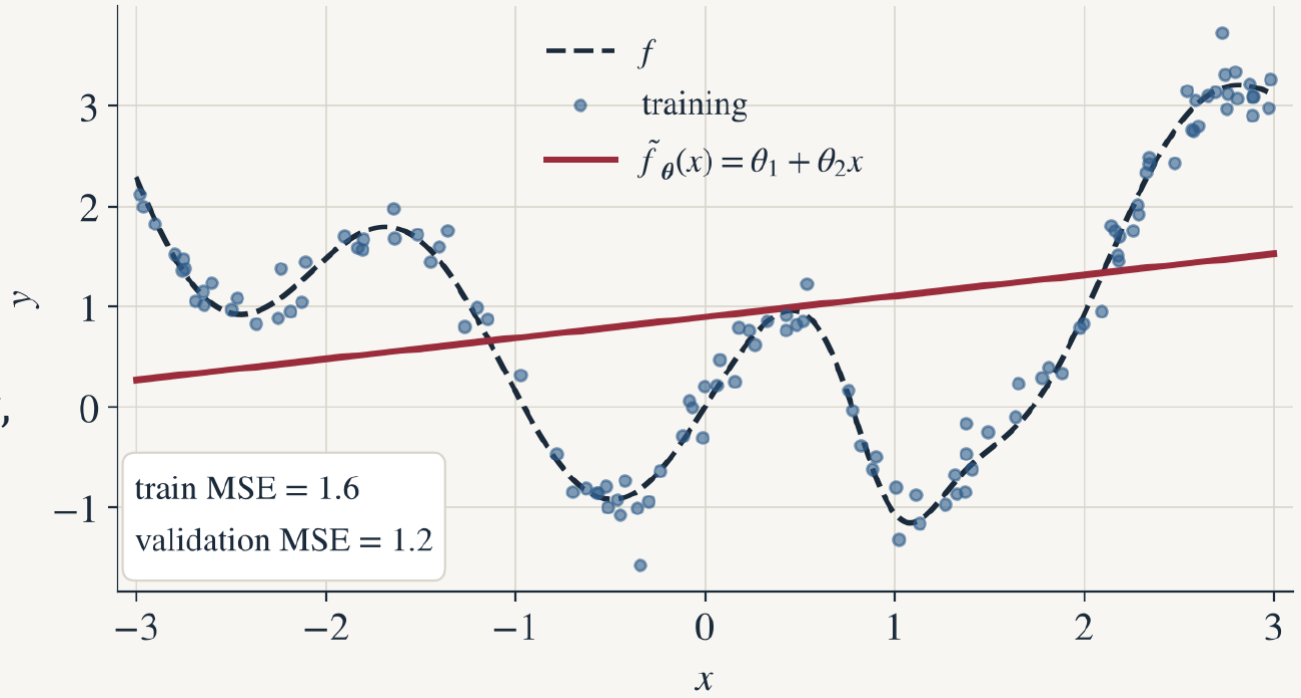
Parametric model: $\tilde{f}$ is chosen in a family indexed by $\boldsymbol{\theta} \in \mathbb{R}^d$.

I. A first model: linear least squares

$$\tilde{f}_{\boldsymbol{\theta}}(x) = \theta_1 + \theta_2\, x, \qquad \boldsymbol{\theta} = (\theta_1, \theta_2) \in \mathbb{R}^2$$

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^2} \frac{1}{N}\sum_{i=1}^{N} (\theta_1 + \theta_2 x_i - y_i)^2 = (X^\top X)^{-1} X^\top \boldsymbol{y},$$

$$X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_N \end{pmatrix}$$



1 · **Model** f̃ with parameters **θ** → part II (MLP)

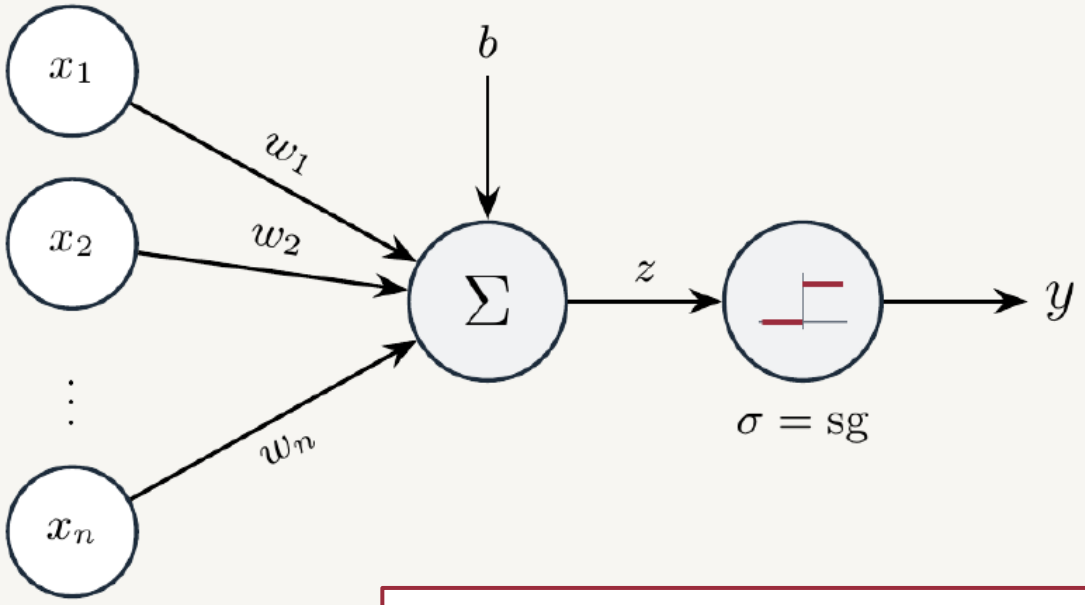
2 · **Loss** measuring the residuals → part III

3 · **Optimizer** to find optimal **θ** → part IV

Example 1, 120 training points: **θ** * = (0.90, 0.21)

Underfitting: Model too simple!

II. From the perceptron [2] to the neural unit



$$z = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{j=1}^{n} w_j \, x_j + b,$$

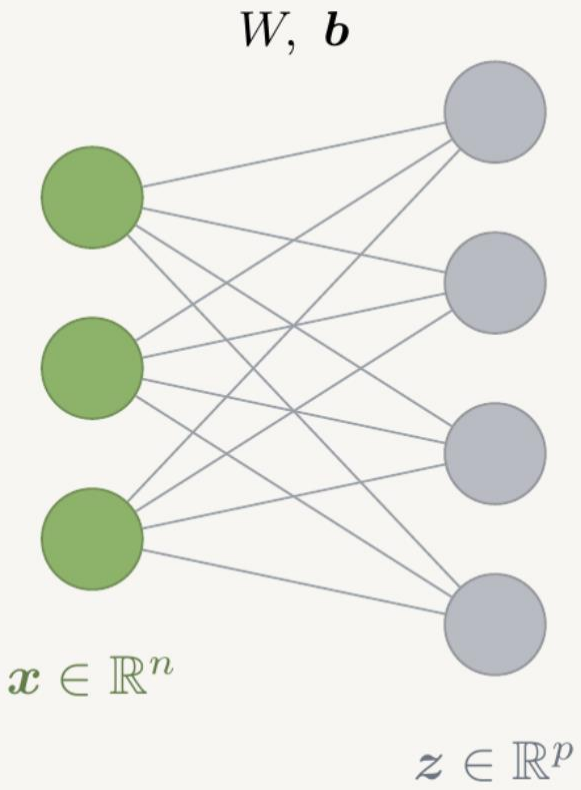
$$y = \mathrm{sg}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$y = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$$

Neural unit: sg → σ nonlinear, differentiable

[2] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, Psychological Review, 1958.

II. One layer

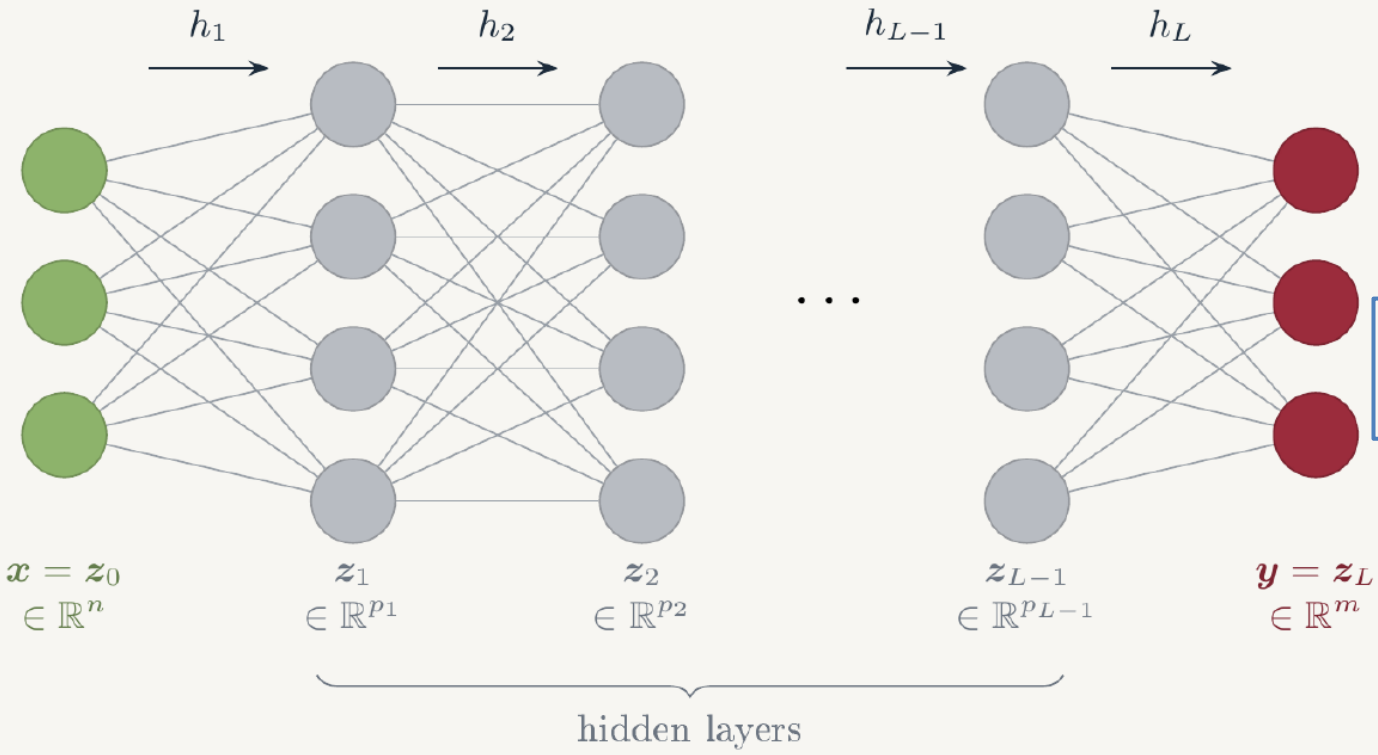


Linear(n,p) -> Activation σ

$$\boldsymbol{z} = \sigma(W\boldsymbol{x} + \boldsymbol{b}) \in \mathbb{R}^p, \qquad W \in \mathbb{R}^{p \times n}, \quad \boldsymbol{b} \in \mathbb{R}^p$$

1 layer: p(n + 1) parameters

II. The multilayer perceptron (MLP): a composition of layers



Names: MLP, FCN (Fully Connected Network)

$$h_\ell(\boldsymbol{z}) = \sigma_\ell(W_\ell\, \boldsymbol{z} + \boldsymbol{b}_\ell),\ W_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}},\ \boldsymbol{b}_\ell \in \mathbb{R}^{p_\ell}$$
$$\text{with } p_0 = n, \quad p_L = m$$

$$\boldsymbol{y} = \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) = (h_L \circ h_{L-1} \circ \cdots \circ h_1)(\boldsymbol{x})$$
$$\rightarrow \boldsymbol{\theta} = \{W_\ell,\ \boldsymbol{b}_\ell\}_{\ell=1}^{L}$$

Number of parameters:

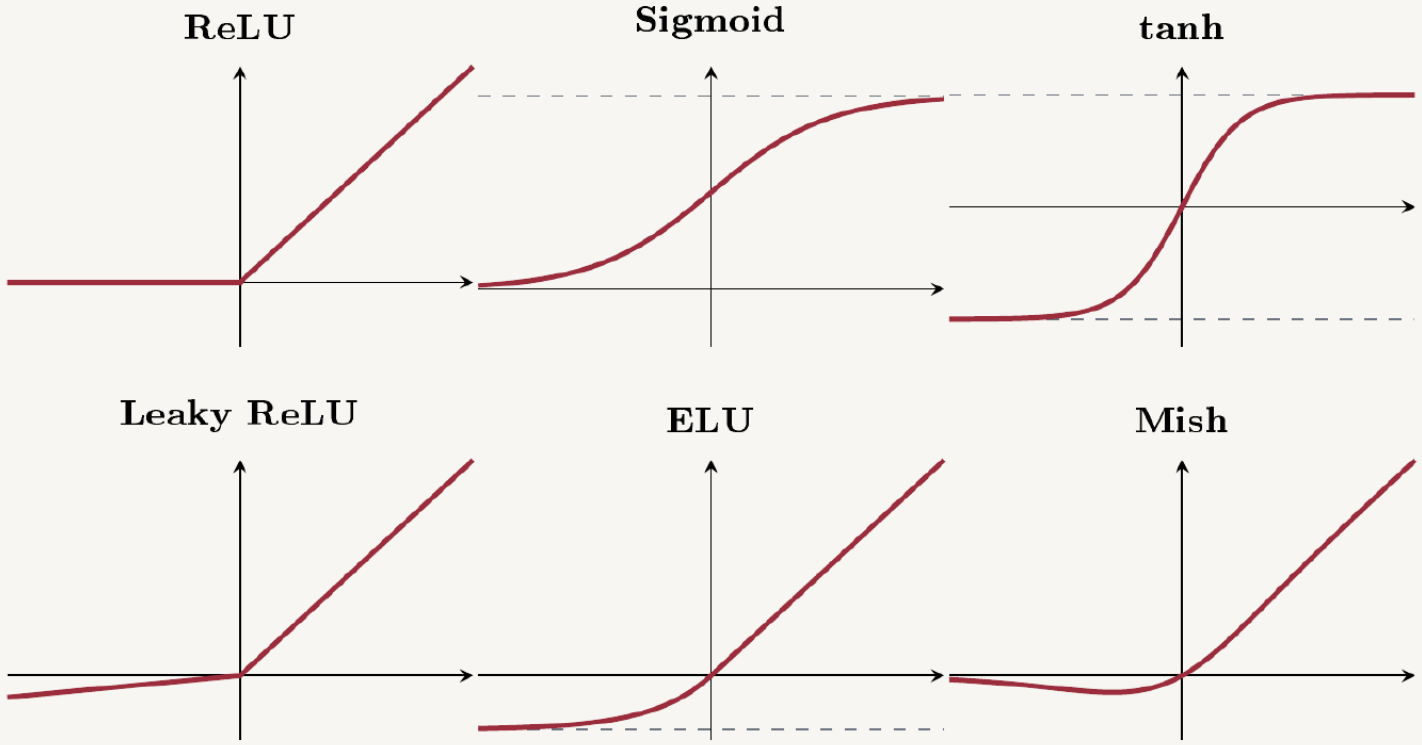
$$d = \sum_{\ell=1}^{L} p_\ell\,(p_{\ell-1} + 1)$$

L layers: **Depth** L -> layers 1 to L − 1 are *hidden*
widths $p_1, \ldots, p_{L-1}$: **number of neurons**

Last layer for regression: σ_L = id.

Hyperparameters (chosen, not learned):
L, the widths, the activations.

II. Activation functions σ



σ	σ(z)	range
ReLU	max(0, z)	[0, +∞[
sigmoid	1 / (1 + e^{-z})	]0, 1[
tanh	(e^{z} − e^{-z}) / (e^{z} + e^{-z})	]−1, 1[
Leaky ReLU	max(αz, z), α ≈ 10^{-2}	$\mathbb{R}$
ELU	z, or e^{z} − 1 if z < 0	]−1, +∞[
Mish	z tanh(ln(1 + e^{z}))	[≈ −0.31, +∞[

Leaky ReLU [3], ELU [4], Mish [5]

[3] Maas et al., ICML Workshop 2013. [4] Clevert et al., ICLR 2016. [5] Misra, BMVC 2020.

II. Why MLPs? The universal approximation theorem [6–8]

Theorem (universal approximation). Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and *not a polynomial*, and $K \subset \mathbb{R}^n$ compact. For every continuous $f : K \to \mathbb{R}^m$ and every $\varepsilon > 0$, there exist a width p and $W \in \mathbb{R}^{p \times n}$, $\mathbf{b} \in \mathbb{R}^p$, $V \in \mathbb{R}^{m \times p}$ such that

$$\sup_{x \in K} \| f(x) - V\, \sigma(Wx + b) \| < \varepsilon.$$

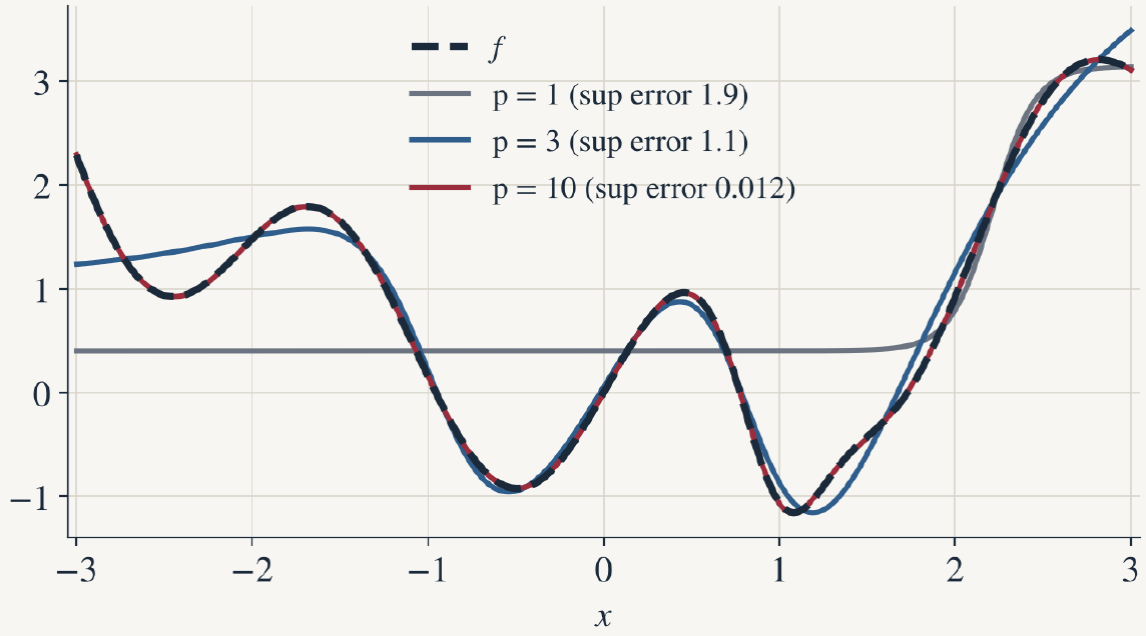


Illustration: one hidden layer of p tanh neurons fitted to f. With p = 10 the network matches f.

Nonlinearity is essential: σ = id is a polynomial, excluded.

Existence only: says nothing about the width p needed, how to find **θ**, or generalization.

Depth: some functions need exponentially many neurons with one hidden layer, few with more layers (Telgarsky; Eldan & Shamir, COLT 2016).

[6] Cybenko, Math. Control Signals Syst., 1989. [7] Hornik, Neural Networks, 1991. [8] Leshno, Lin, Pinkus, Schocken, Neural Networks, 1993.

III. The loss function

$f: \mathbb{R}^n \to \mathbb{R}^m, \qquad \boldsymbol{x} \mapsto \boldsymbol{y} = f(\boldsymbol{x})$ unknown

$\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N, \qquad \boldsymbol{y}_i = f(\boldsymbol{x}_i) + \boldsymbol{\varepsilon}_i$ (noise)

find $\boldsymbol{\theta} \in \mathbb{R}^d$ such that $\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx f(\boldsymbol{x})$

Regression loss: Mean Square Error (MSE)

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i \|_2^2$$

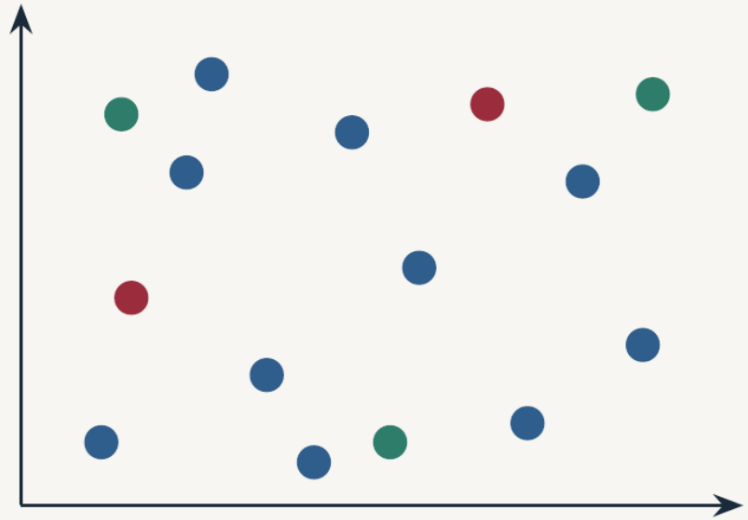
Loss = cost function = *empirical risk*

III. Data: normalize, then split

1 · Normalize inputs and outputs to order 1:

$$\tilde{x} = \frac{x - \mu}{s}$$

2 · Split at random into three disjoint sets:



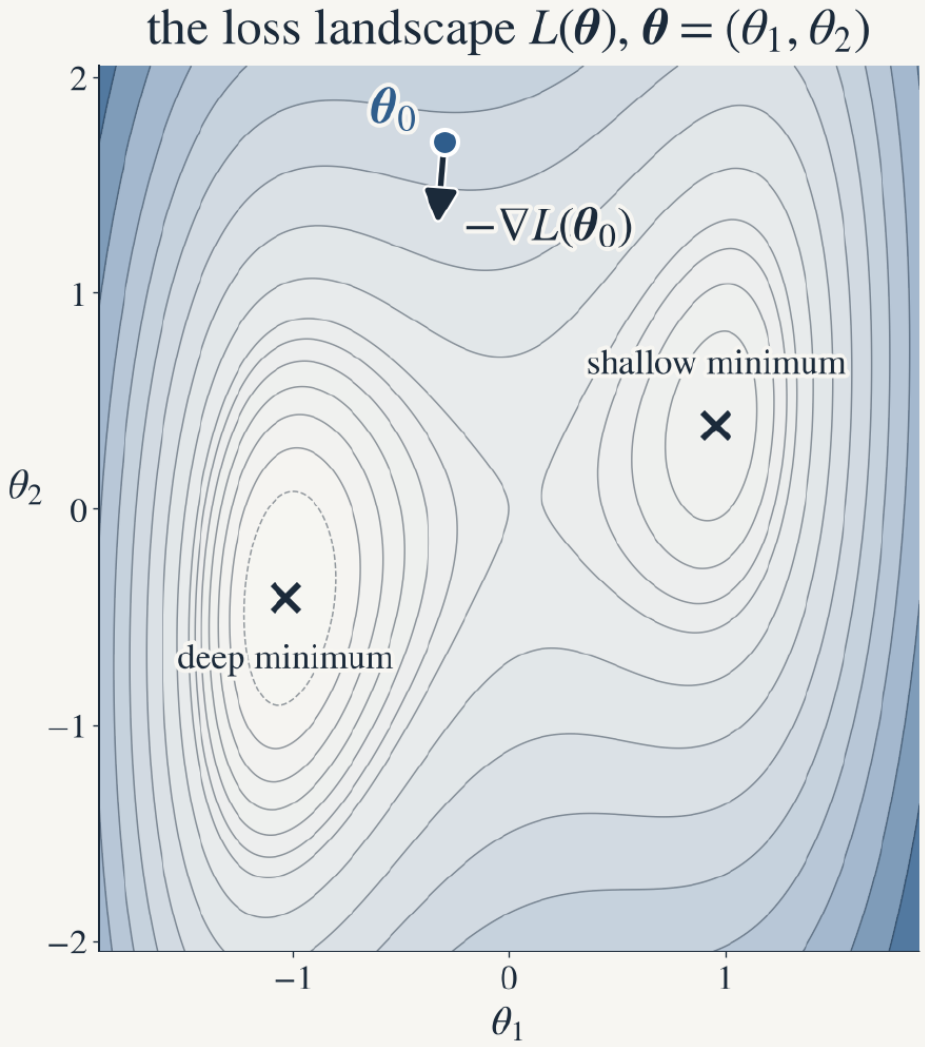
- **Training (≈ 60 %)**
  used to **fit θ**: we minimize the loss on it

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} \| \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i \|_2^2$$

- **Validation (≈ 20 %)**
  never updates θ: detects overfitting, chooses hyperparameters

- **Test (≈ 20 %)**
  used **once**, at the very end: final error

III. Training is an optimization problem



$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$$

- **d unknowns**: 10^3 (a PINN) … 10^{11} (GPT-3)
- $\mathcal{L}$ is **nonlinear** in θ: **no formula** for θ*
- so we **iterate**: $\theta_0 \rightarrow \theta_1 \rightarrow \theta_2 \rightarrow \ldots \rightarrow \theta^*$
- $\mathcal{L}$ has **several valleys**: the result depends on θ_0

IV. Reminder: a minimum is a critical point



$$\theta^* \text{ minimum } \Rightarrow \mathcal{L}'(\theta^*) = 0$$

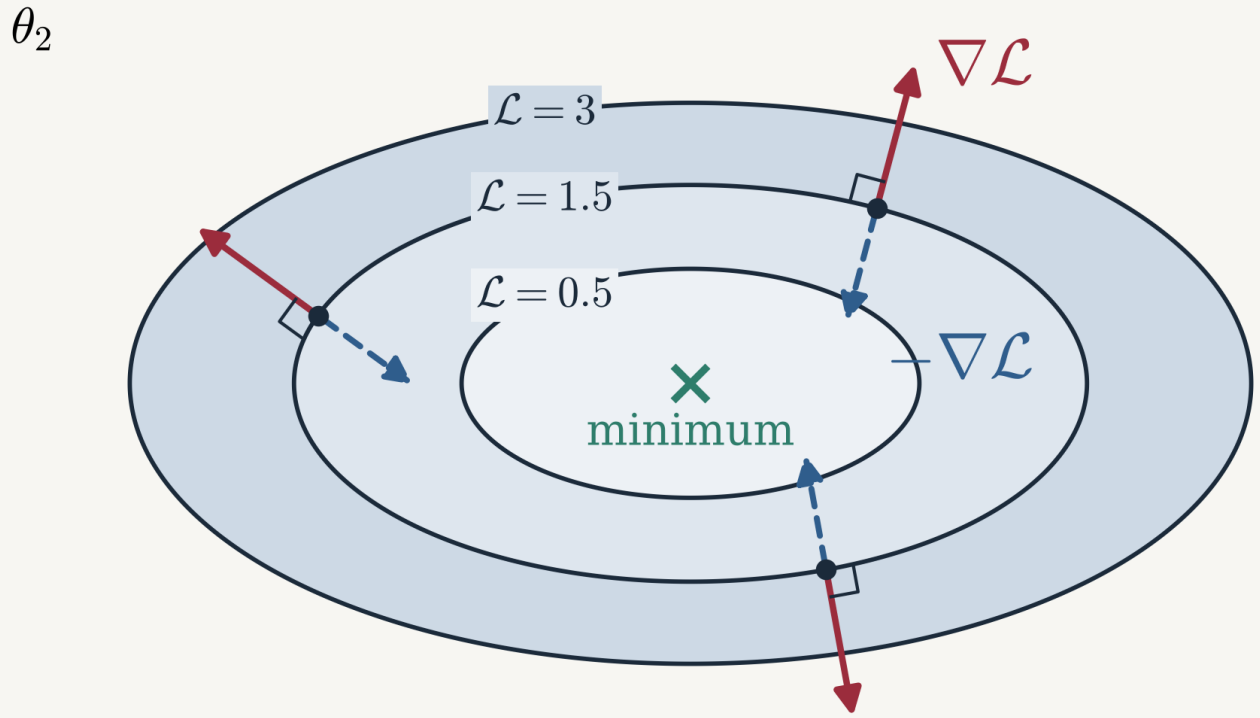
Necessary, **not sufficient**: a critical point can be a minimum, a maximum or a saddle point.

In d dimensions:

$$\nabla\mathcal{L}(\boldsymbol{\theta}^*) = \mathbf{0}, \qquad \nabla\mathcal{L} = \left(\frac{\partial\mathcal{L}}{\partial\theta_1}, \ldots, \frac{\partial\mathcal{L}}{\partial\theta_d}\right)$$

A **local** minimum is not always the **global** one.

IV. The gradient is perpendicular to the level lines



- Level line: all θ with the **same loss** $\mathcal{L}(\theta) = c$
- $\nabla\mathcal{L}$ is **perpendicular** to the level lines and points to **larger** values
- $-\nabla\mathcal{L}$: the direction of **steepest descent**

Why? Taylor, for a small step h:

$$\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{h}) \approx \mathcal{L}(\boldsymbol{\theta}) + \nabla\mathcal{L}(\boldsymbol{\theta}) \cdot \boldsymbol{h}$$

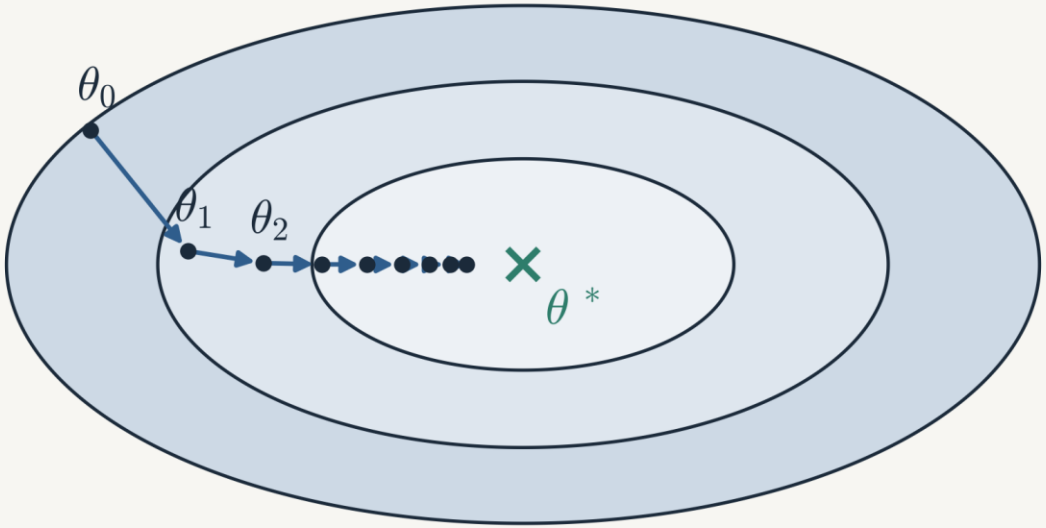
Like electrostatics: equipotentials V = c, and the field **E** = −∇V perpendicular to them.

IV. Gradient descent

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

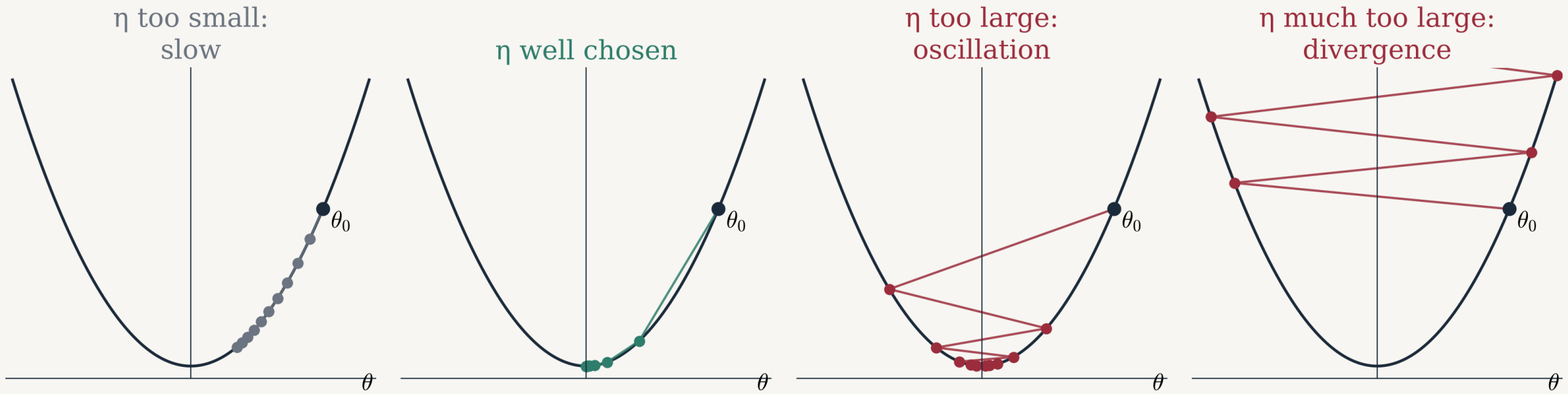
η > 0: the learning rate (step size)

- $\boldsymbol{\theta_0}$: random (PyTorch does it)
- $\boldsymbol{\nabla \mathcal{L}(\theta_k)}$: computed exactly by automatic differentiation: loss.backward()
- repeat until $\mathcal{L}$ stops decreasing



IV. Choosing η: not too small, not too large

Model problem $\mathcal{L}(\theta) = \theta^2$, same start θ_0, 10 steps:

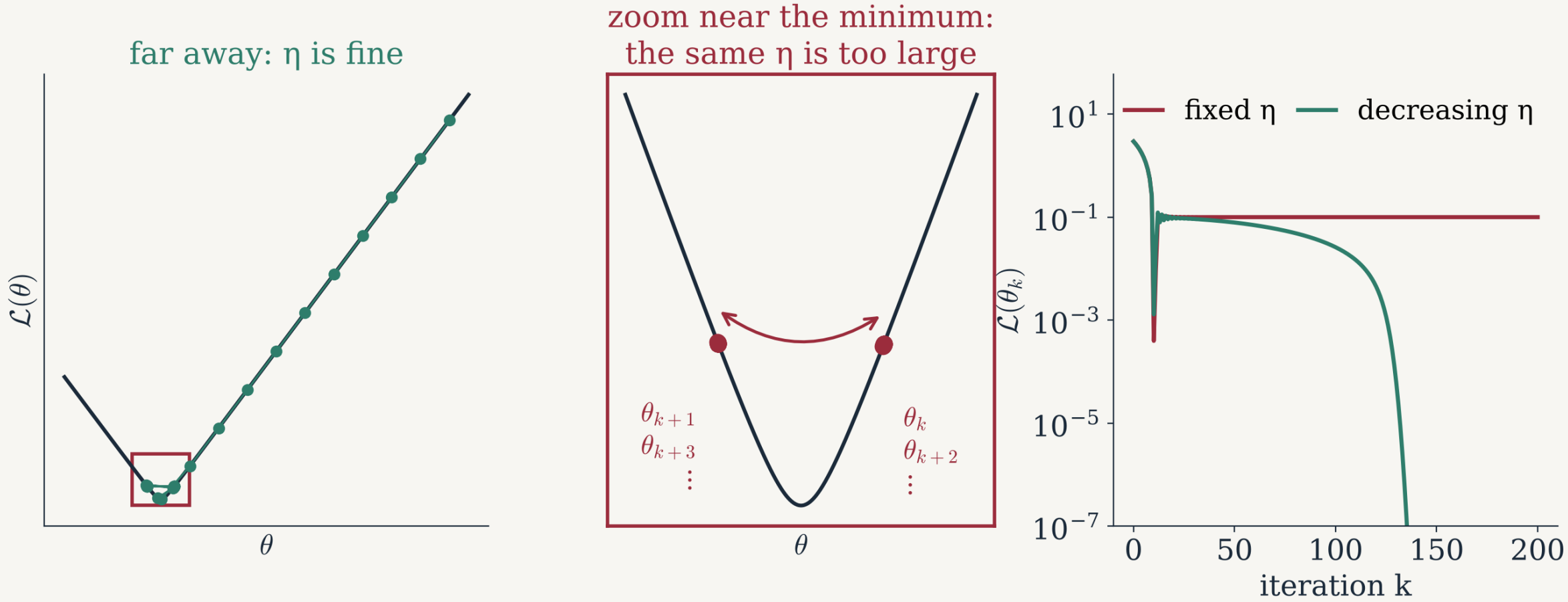


$$\theta_{k+1} = \theta_k - \eta \cdot 2\theta_k = (1 - 2\eta)\,\theta_k$$

converges $\Leftrightarrow$ $0 < \eta < 1$; **alternates** sides for $\eta > ½$; **diverges** for $\eta > 1$

Too small: slow. Too large: oscillation, then divergence.

IV. Even a good η is too large near the minimum



Near θ*, the steps must shrink: decrease η during training (learning-rate scheduler).

IV. Learning-rate schedulers: large steps, then small steps

step decay

$$\eta_k = \eta_0 \, \gamma^{\lfloor k/s \rfloor}$$

```
StepLR(opt, step_size=s, gamma=γ)
```

exponential

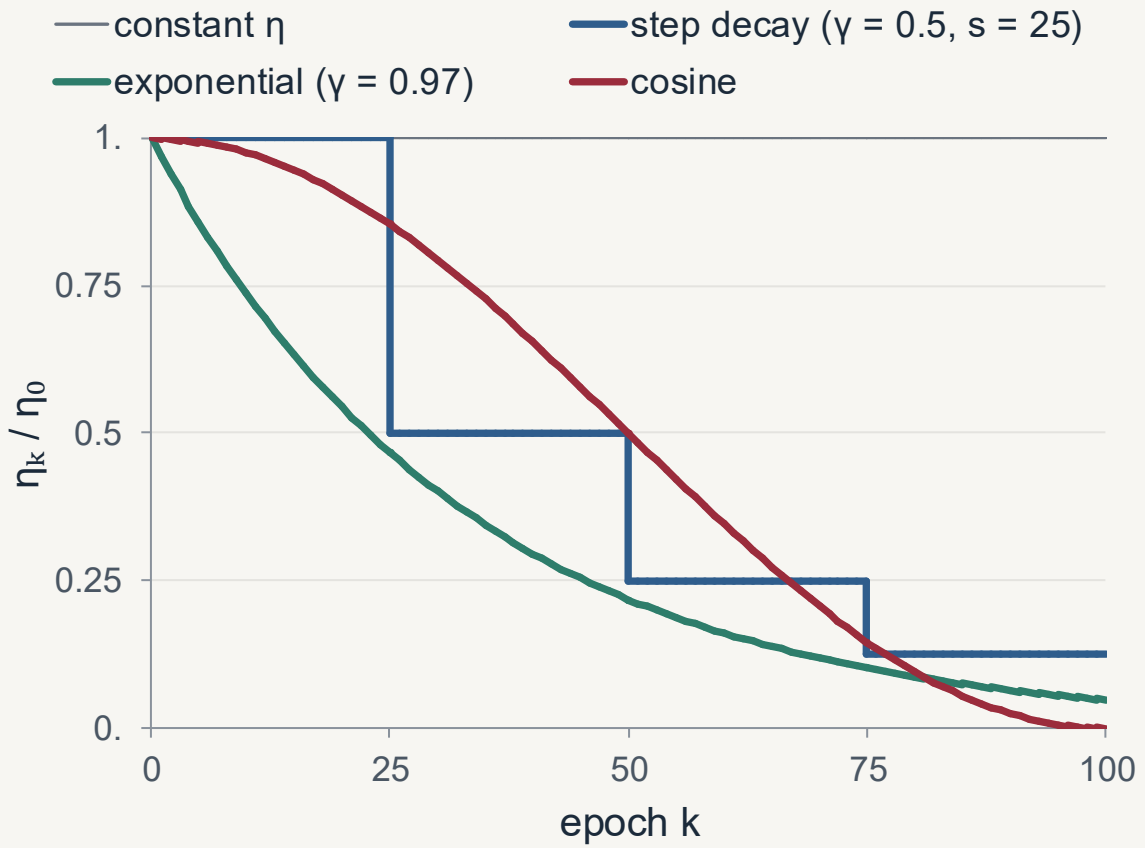
$$\eta_k = \eta_0 \, \gamma^{k}$$

```
ExponentialLR(opt, gamma=γ)
```

cosine

$$\eta_k = 1/2 \; \eta_0 \left(1 + \cos \frac{\pi k}{K}\right)$$

```
CosineAnnealingLR(opt, T_max=K)
```



In PyTorch: **scheduler.step()** once per epoch, after the optimizer steps.

IV. Adam: the default optimizer [14]

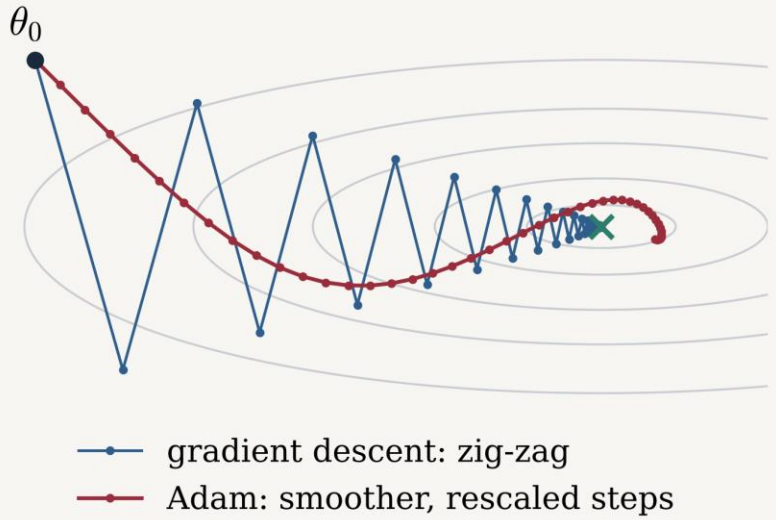
$$\boldsymbol{g}_k = \nabla \mathcal{L}(\boldsymbol{\theta}_{k-1})$$ gradient

$$\boldsymbol{m}_k = \beta_1 \boldsymbol{m}_{k-1} + (1-\beta_1)\, \boldsymbol{g}_k$$ average of the gradients

$$\boldsymbol{v}_k = \beta_2 \boldsymbol{v}_{k-1} + (1-\beta_2)\, \boldsymbol{g}_k^2$$ average of their squares

$$\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} - \eta \frac{\hat{\boldsymbol{m}}_k}{\varepsilon + \sqrt{\hat{\boldsymbol{v}}_k}}$$ step per parameter

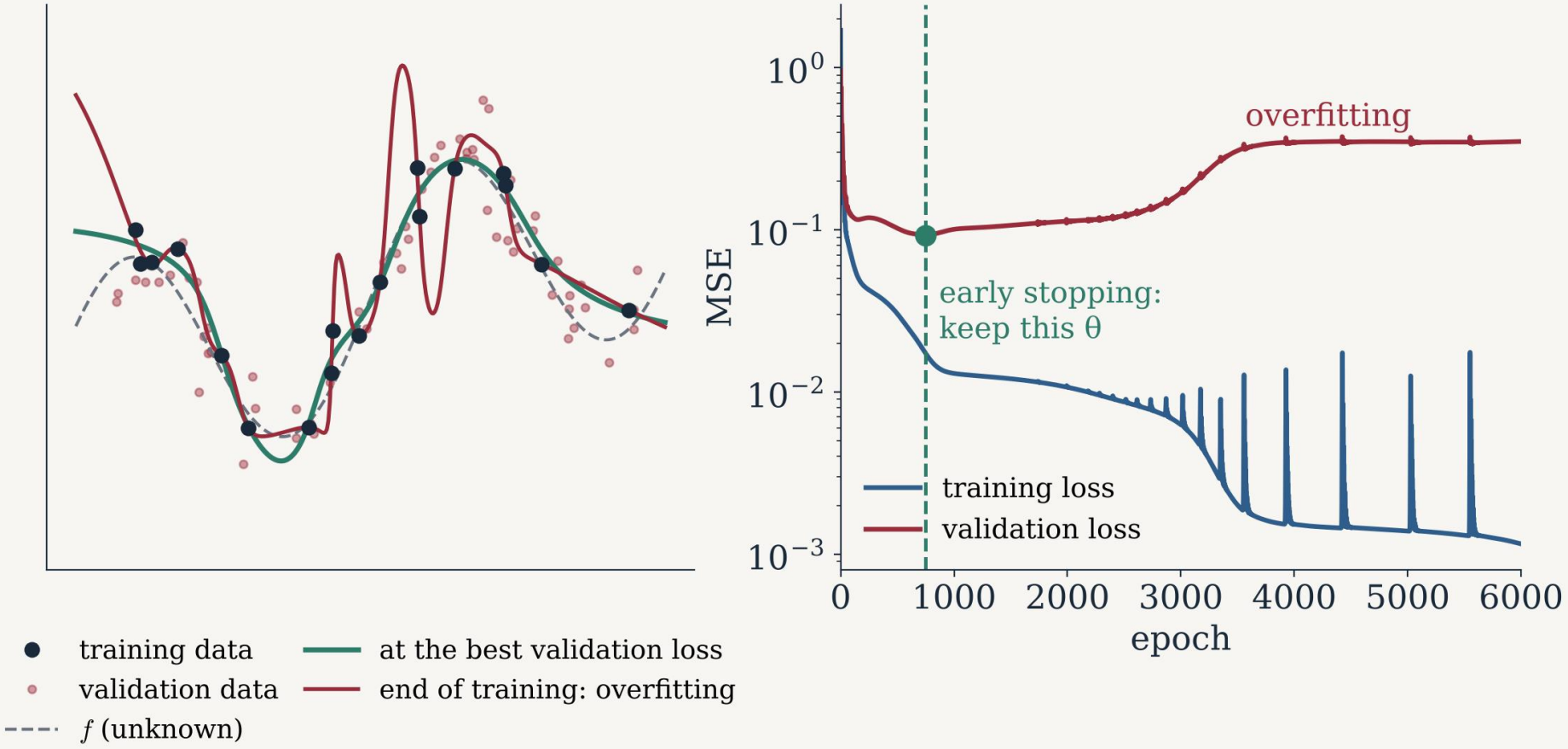
$\hat{m}$, $\hat{v}$: m, v divided by $(1-\beta^k)$ (bias correction). $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$



Same rules for η: not too small, not too large, decrease it near the minimum. Default: $\eta = 10^{-3}$.

[14] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, ICLR, 2015. PyTorch: torch.optim.Adam(model.parameters(), lr=η)

V. Overfitting: watch the validation loss

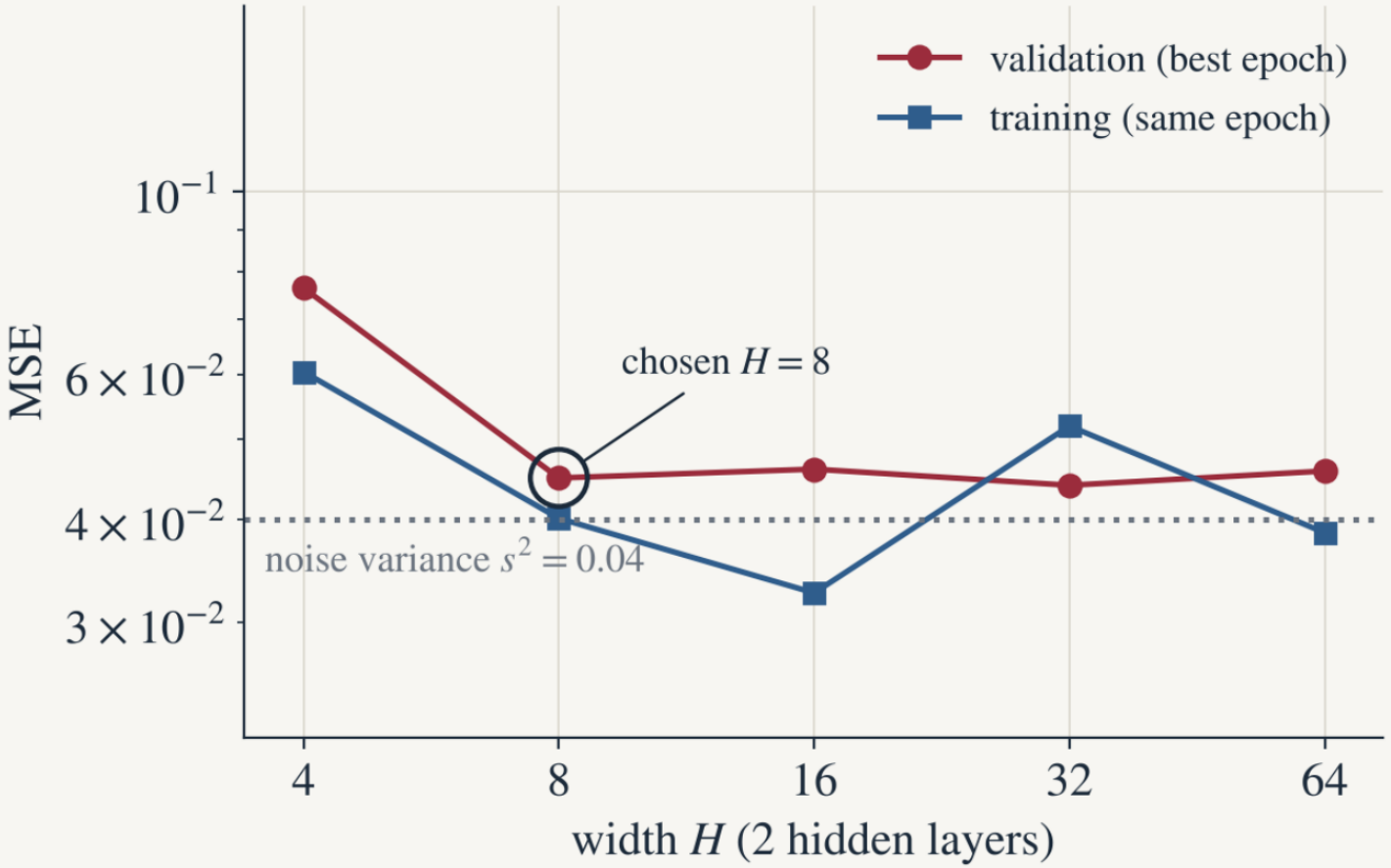


Training loss always goes down

Validation loss goes up: **overfitting**

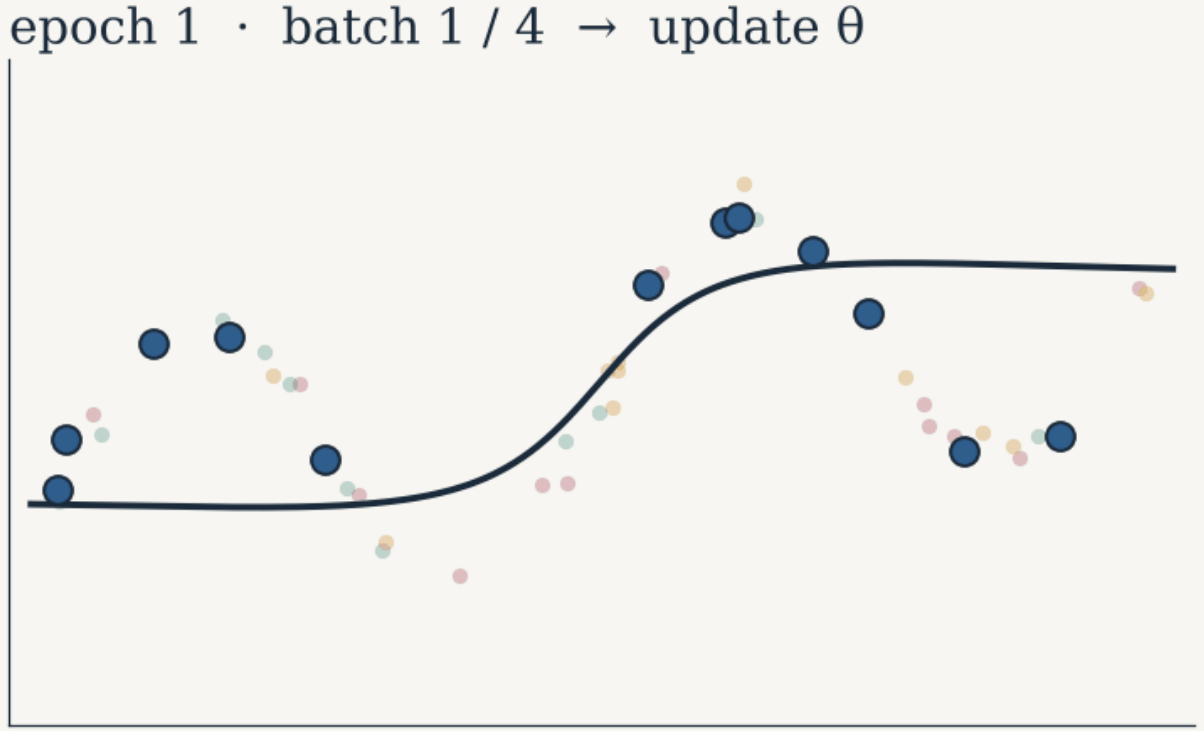
Early stopping: keep θ at the lowest validation loss

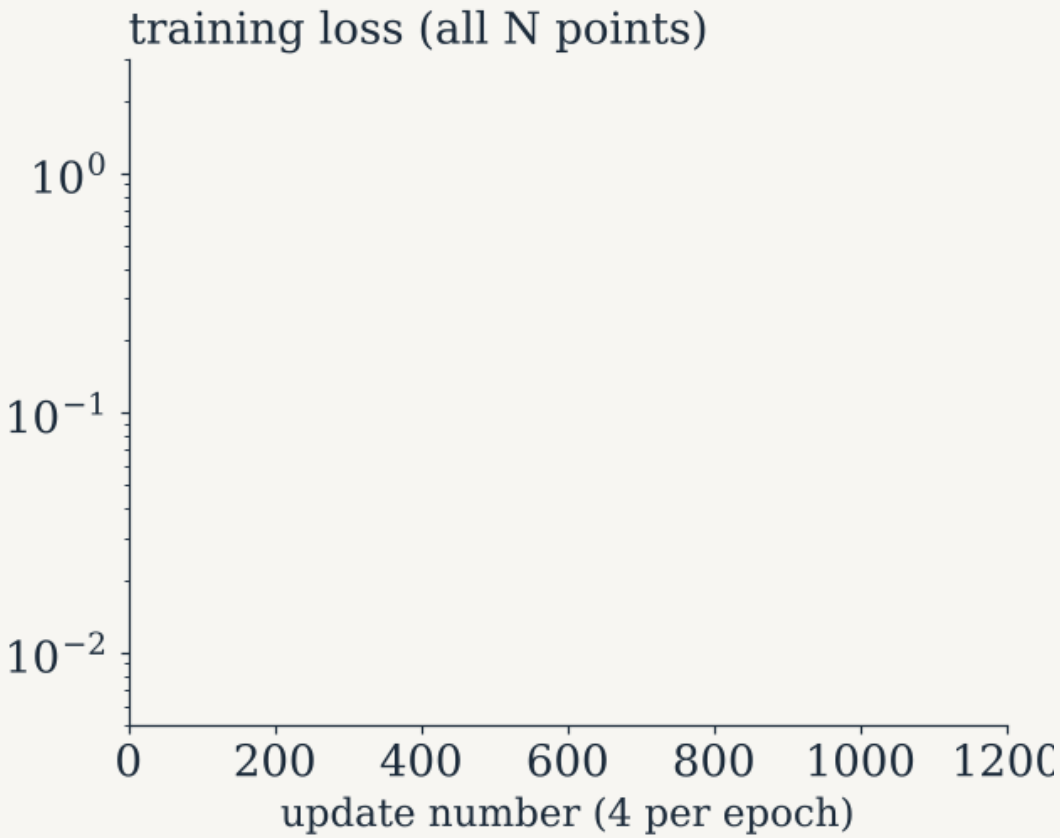
V. Hyperparameters: the validation set decides



- **Hyperparameter**: chosen before training (width, depth, σ, η, batch size…)
- Training error always prefers the **largest** network
- Choose the **smallest** network with a good **validation** error (here H = 8)
- **Test** error: once, at the end

V. Mini-batches: update θ after each small packet



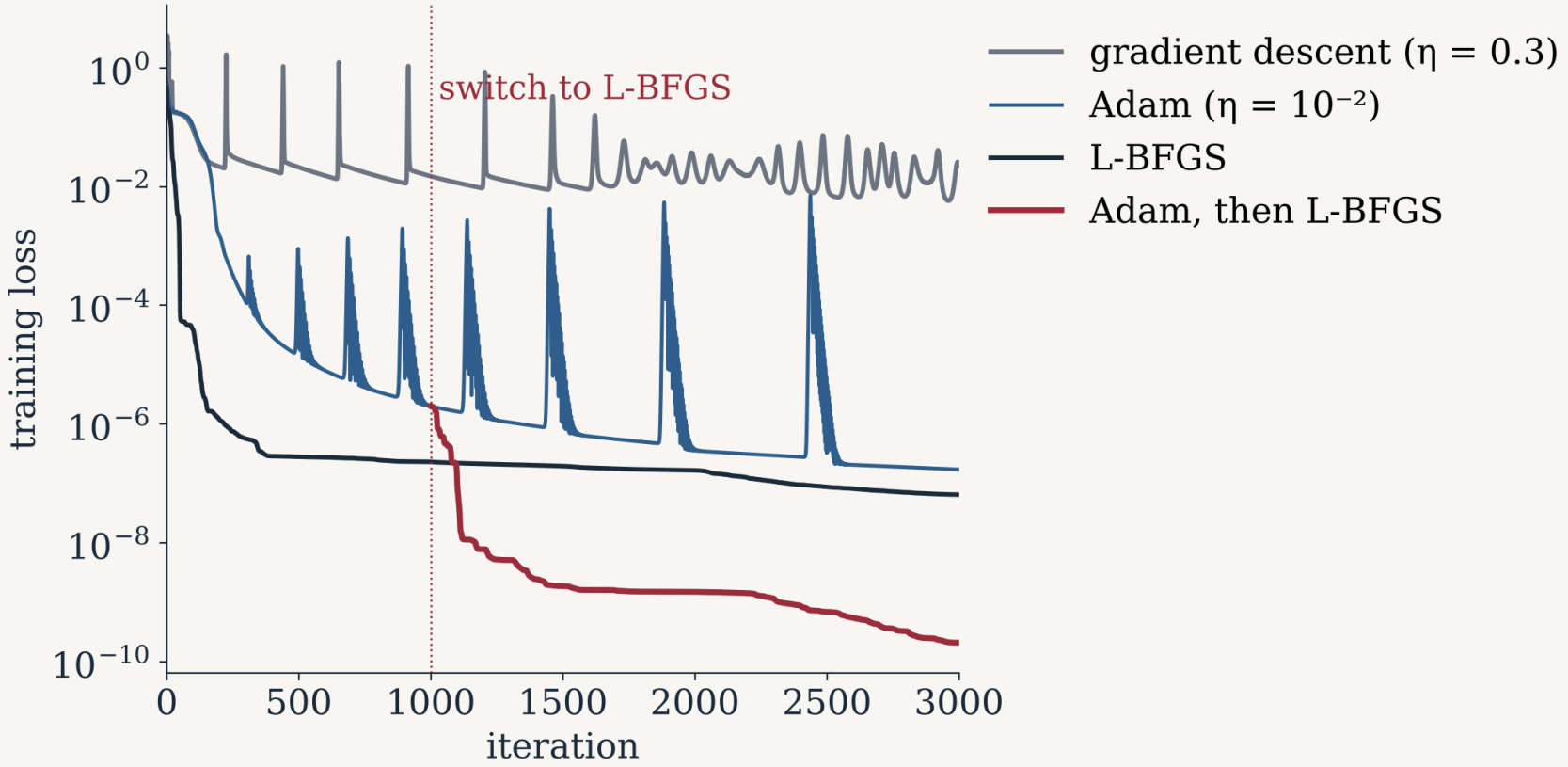


$$\mathcal{L}_{\text{batch}}(\boldsymbol{\theta}) = \frac{1}{B} \sum_{i \in \text{batch}} \|\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\|^2, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}_{\text{batch}}(\boldsymbol{\theta})$$

1 epoch = one pass = **N / B updates**

noisier steps, but many more of them

V. For PINNs: Adam first, then L-BFGS



- **L-BFGS** also uses the **curvature**, estimated from the last gradients
- no η to tune: line search
- full batch only
- torch.optim.LBFGS

sin(2πx), 256 points, MLP $1 \to 20 \to 20 \to 1$ ($d = 481$), full batch, same θ_0.

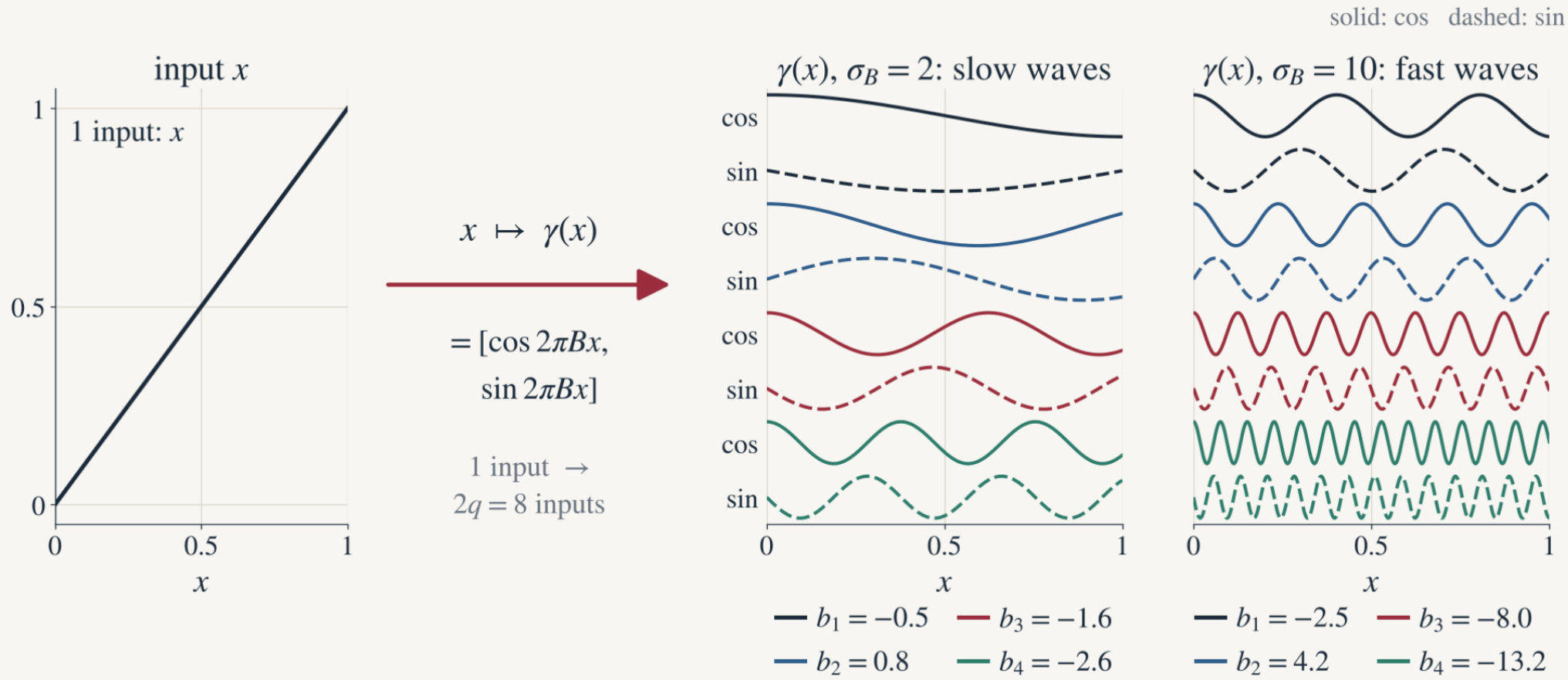
V. Everything in one loop (PyTorch)

```
model = MLP([1, 64, 64, 1])
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) # lr = η
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, n_epochs)

for epoch in range(n_epochs):
    for xb, yb in train_loader: # mini-batches
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb) # loss on the batch
        loss.backward() # gradient (autodiff)
        optimizer.step() # update θ
    scheduler.step() # decrease η
    with torch.no_grad():
        val_loss = loss_fn(model(X_val), Y_val) # watch overfitting
```

Then, for PINNs: a few hundred **L-BFGS** iterations on the full batch.

V. Fourier features: feed the network with waves

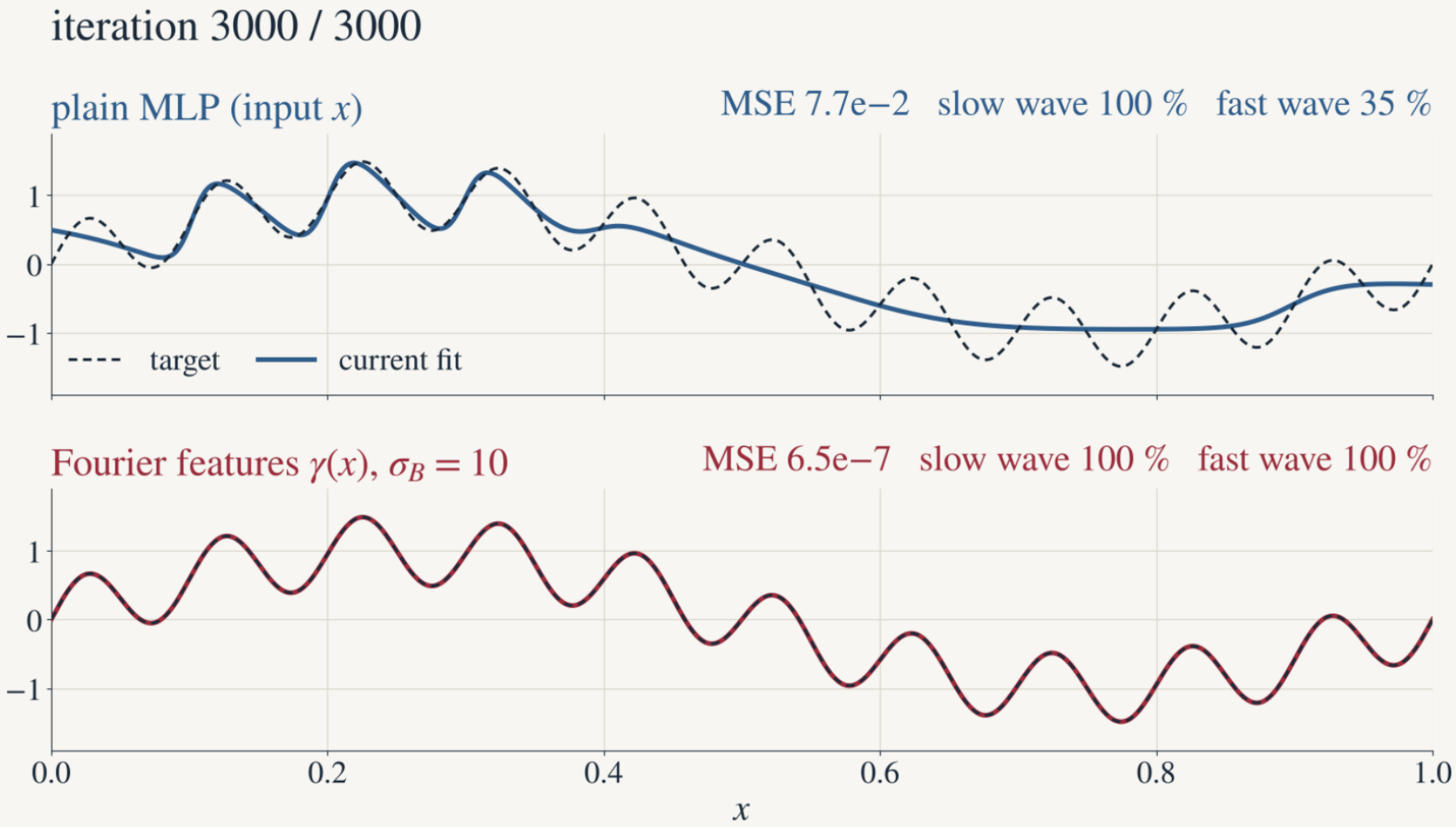


$$\gamma(\boldsymbol{v}) = [\cos(2\pi B\boldsymbol{v}),\ \sin(2\pi B\boldsymbol{v})], \qquad B_{ij} \sim \mathcal{N}(0, \sigma_B^2)$$

B random, fixed. σ_B: how fast the waves are (a hyperparameter).

[22] M. Tancik et al., Fourier features let networks learn high frequency functions in low dimensional domains, NeurIPS, 2020.

V. Effect: the fast wave is learned at once



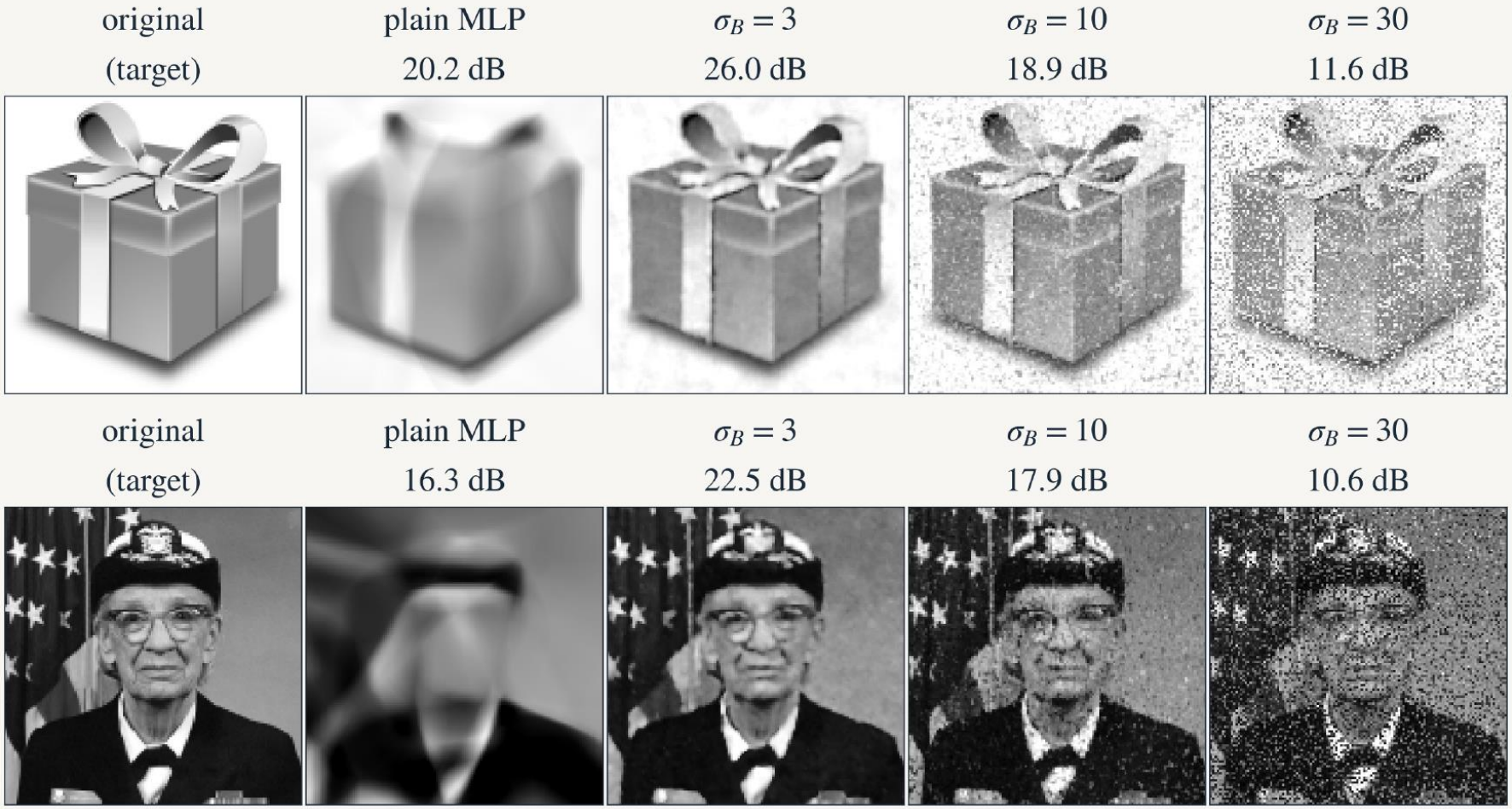
Plain MLP: slow wave first; fast wave still missing after 3000 iterations

Fourier features: both waves within 50 iterations

Networks learn **low frequencies first**: "spectral bias" [21]

[21] N. Rahaman et al., On the spectral bias of neural networks, ICML, 2019.

V. On images: sharper, but σ_B must be chosen



- input (x, y) → grey level
- σ_B = 3: sharp
- σ_B = 10, 30: noise between pixels: **overfitting**
- σ_B chosen on **validation**

NEXT: HANDS-ON

Now you train your own networks, in a notebook

We practice everything seen today with PyTorch, step by step. The same code will be the starting point of our PINNs.

